Campos de Bits
Uniones y
Enumeraciones

Campos de Bits - Introducción

- *El lenguaje "C" dispone de un método de acceso para los bits individuales de una palabra.*
- *Esto es especialmente útil cuando tenemos que tratar con registros cuyos bits individuales tienen un significado particular.*
- *Los campos de bits permiten nominar cada bit de una palabra.*

Introducción

- *De esta manera el código resultante es más claro y eficiente.*
- *Un campo de bits es un tipo especial de elementos de estructura que define su tamaño en bits.*
- *Los campos de bits son usados especialemente en el tratamiento con puertos y elementos de comunicación.*

Introducción

- *La forma general de declaración de un campo de bits es:*

```
struct etiqueta (
  tipo nombre1: longitud;
  tipo nombre2: longitud;
  ...
  tipo nombren: longitud;
} lista de variables;
```

- *Si bien la mayoría de los compiladores admiten campos signed, unsigned e int, se deberían declarar como unsigned.*

```
// Ejemplo de campo de bits

#include <stdio.h>

int main( void )
{
  struct modemstatus {
    unsigned  DeltaCTS: 1;
    unsigned  DeltaDSR: 1;
    unsigned   DeltaRI: 1;
    unsigned  DeltaDCD: 1;
    unsigned       CTS: 1;
    unsigned       DSR: 1;
    unsigned        RI: 1;
    unsigned       DCD: 1;
  };

  union bitfil {
    struct modemstatus ms1;
    int inbyte;
  };
```

```
// Ejemplo de campo de bits (Continuación)

  union bitfil codbit, *codbit2;

  codbit.inbyte = 233;

  if( codbit.ms1.DeltaCTS ) printf( "Cambio en Modem Listo\n");
  if( codbit.ms1.DeltaDSR ) printf( "Cambio en Modem Activo\n");
  if( codbit.ms1.DeltaRI  ) printf( "Cambio en llamada de
                                     línea\n" );
  if( codbit.ms1.DeltaDCD ) printf( "Cambio de estado en
                                     portadora\n" );
  if( codbit.ms1.CTS      ) printf( "Modem Listo\n" );
  if( codbit.ms1.DSR      ) printf( "Modem Activo\n" );
  if( codbit.ms1.RI       ) printf( "Línea llamando\n" );
  if( codbit.ms1.DCD      ) printf( "Portadora detectada\n" );
}
```

Campos de Bits

- *Los bits de un campo de bits se asignan igual que los elementos de una estructura, utilizando el punto si es una variable común o el operador flecha si es un apuntador a estructura.*
- *No es necesario inicializar todos los bits de un campo de bits, se puede hacer solo con los necesarios.*

Declaración de Campos de Bits

```
7     6     5     4     3     2     1     0
    baudios       paridad       ↑    bits de datos
                                bits de parada
```

```
struct linestatus {
    unsigned   baudios: 3;
    unsigned   paridad: 2;
    unsigned    parada: 1;
} linea1;
```

Si los que queremos están al medio tenemos:

```
struct linestatus {
    unsigned           : 3;
    unsigned   paridad: 2;
} linea2;
```

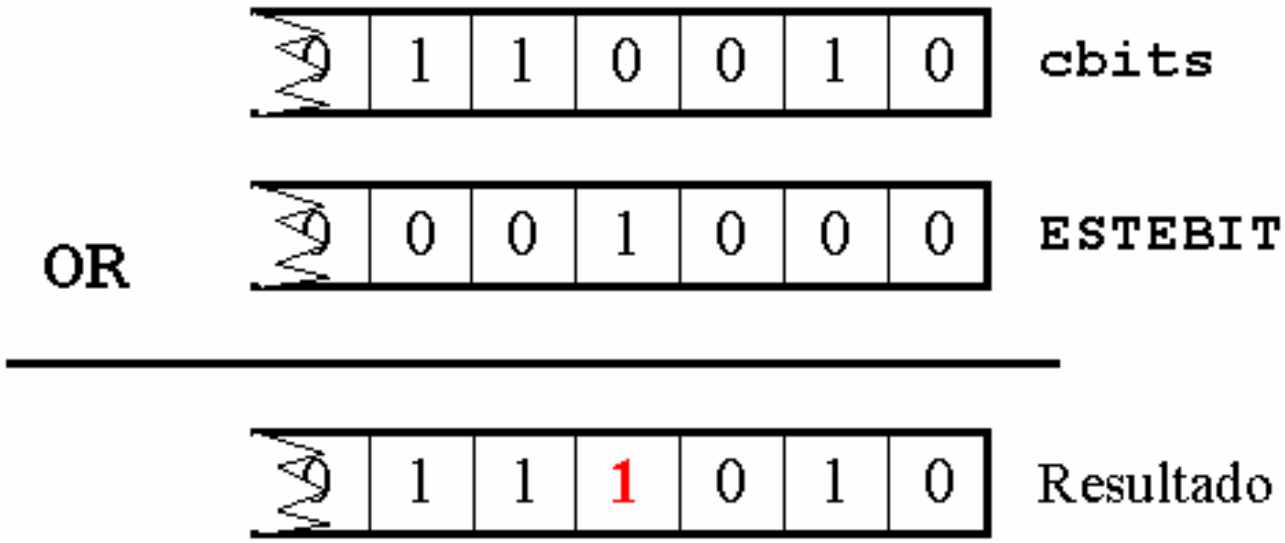
Campos de Bits

- *No se puede averiguar la dirección de una variable de campo de bits.*
- *No se pueden construir arreglos de variables de campos de bits.*
- *Puede haber problemas de portabilidad con los campos de bits, especialmente con little y big endian.*
- *Se pueden mezclar los campos de bits con elementos de estructura normales.*

Operaciones con Campos de Bits

- *Activación de un bit sin afectar el resto.*

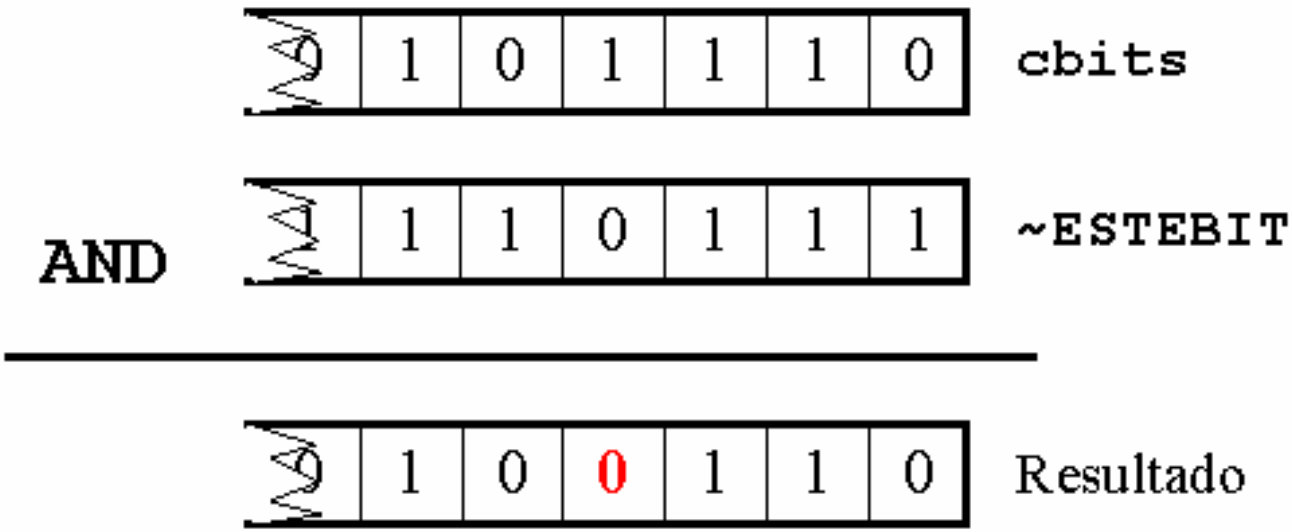
```
cbits = cbits | ESTEBIT;
```



Operaciones con Campos de Bits

- *Desactivación de un bit sin afectar el resto.*

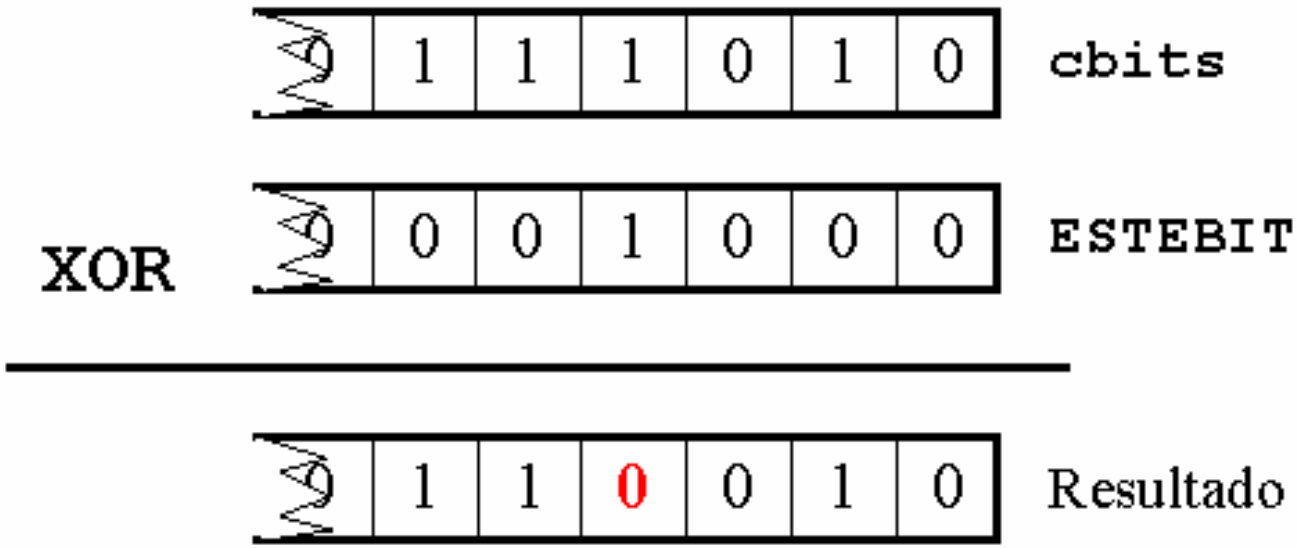
```
cbits = cbits & ~ESTEBIT;
```



Operaciones con Campos de Bits

- *Cambiar el estado de un bit sin afectar el resto.*

```
cbits = cbits ^ ESTEBIT;
```



Uniones

- *En C una unión es una posición de memoria que es compartida por dos o más variables diferentes, generalmente de distinto tipo y en distintos momentos.*
- *La definición de una unión es similar a la de una estructura.*
- *Un uso frecuente de uniones es con campos de bits.*

Uniones

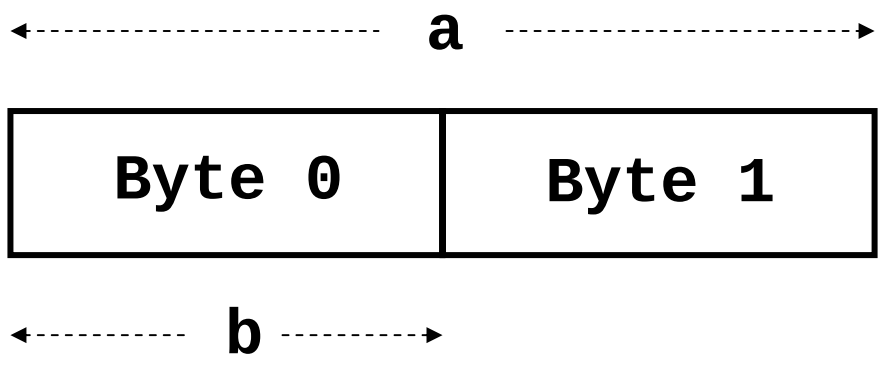
```
union etiqueta {
  tipo nombrevariable1;
  tipo nombrevariable2;
  ...
  tipo nombrevariablen;
} variables de unión;
```

Ejemplo:

```
union ejemplo {
  int a;
  char b;
};
union ejemplo c;
```

Uniones

- *Las variables comparten el almacena-miento de la siguiente forma:*



Uniones

- *Al hacer la declaración de unión el compilador crea un área de almacena-miento de igual tamaño a la más grande de las variables declaradas.*
- *Las uniones generalmente son indepen-dientes de la arquitectura de la máqui-na. También proveen conversión auto-mática de tipos.*

Uniones

- *También se puede realizar una unión en algunos elementos con arreglos.*

  Ejemplo:

```
union {
  unsigned base;
  char elem[4];
}
```

Enumeraciones

- *El estándar ANSI incluye la enumeración como una extensión del lenguaje C.*
- *Una enumeración es un conjunto de constantes enteras con nombre que especifica todos los valores válidos que una variable de ese tipo puede tener.*
- *Las enumeraciones son normales en la vida cotidiana.*

Enumeraciones

Ejemplo:

```
#include <stdio.h>

enum planetas { venus, tierra, marte, jupiter, saturno,
                urano, neptuno, pluton };

void PrintPlanet( enum planetas item );

int main( void )
{
  enum planetas sonda;
  for( sonda=venus; sonda<=pluton; sonda++ ) {
    printf( "Está en " );
    PrintPlanet( sonda );
  }
}
```

Enumeraciones

Ejemplo (continuación)

```
void PrintPlanet( enum planetas item) {
  switch( item ) {
    case venus  : printf( "Venus\n" ); break;
    case tierra : printf( "Tierra\n" ); break;
    case marte  : printf( "Marte\n" ); break;
    case jupiter: printf( "Júpiter\n" ); break;
    case saturno: printf( "Saturno\n" ); break;
    case urano  : printf( "Urano\n" ); break;
    case neptuno: printf( "Neptuno\n" ); break;
    case pluton : printf( "Plutón\n" ); break;
    default: printf( "ERROR DE ARGUMENTO\n" ); break;
  };
}
```

Enumeraciones

- *En realidad cada uno de los símbolos corresponde a un valor entero, lo que significa que si consideramos que el primer elemento tiene el valor 0, éstos pueden usarse en operaciones enteras.*
- *Es posible asignar un valor distinto de la escala normal a cualquiera de los elementos y los siguientes se contarán agregando uno al anterior.*

Enumeraciones

- *Ejemplo:*

```
enum planetas { venus, tierra=10, marte, jupiter,
                saturno, urano, neptuno, pluton };
```

- *En este caso marte tendrá el valor 11, jupiter 12 y así sucesivamente.*
- *Venus sigue teniendo el valor 0.*

Definiciones de Tipos

- *Esto es simplemente el cambio de nombre a un tipo existente de datos, ej.:*

```
typedef tipo nombre;
typedef double carga;
```

- *De aquí en adelante se puede usar* **`carga`** *en lugar de* **`double`**.
- *Es posible utilizar* **`carga`** *para un nuevo typedef.*